Stellar Cyber 6.6.0s is now live! Discover all the updates in the release notes!

ATH Example: Reset TCP Connections after Detecting Threats

Create an Automated Threat Hunting (ATH) playbook that terminates TCP connections by sending TCP reset packets to both ends of a connection. This enables Stellar Cyber to disrupt malicious connections automatically as soon as it detects a threat.

To use this feature, you must select a Modular Sensor that has network visibility into the affected traffic path. The Modular Sensor must be positioned on a network segment where it can observe the relevant packets and send TCP resets to the source and destination IP addresses involved in detected threats.

The following are a few cases where quickly terminating the TCP connection reduces harm:

- **Exploit attempts with high-confidence signatures**

  Example: If Stellar Cyber detects clear IDS/IPS signatures for protocol exploits, such as an HTTP request that matches a known remote-code-execution exploit, terminating the connection prevents the delivery of an exploit payload or code execution staging.

- **Confirmed Command-and-Control (C2) callbacks**

  Example: If Stellar Cyber detects small, regular beaconing to known-malicious IP addresses or domain names or to a destination proven to be C2 server, breaking the TCP connection disrupts the attacker's control channel.

- **Active data-exfiltration over TCP with DLP match**

  Example: If there's a file transfer where Data Loss Prevention (DLP) rules match sensitive data being sent to an external host, immediately terminating the TCP connection limits data loss.

Create a Remote Host Connector

Create the following remote host connector to run the script on a Modular Sensor when the time comes to respond automatically to a detected threat. For more information about configuring a remote host connector, see **Configuring Remote SSH Host Connectors**.

1. Navigate to **System | INTEGRATIONS | Connector**, select **Create | Connector**, and enter the following in the **General** section:

   Category: **remote_host**

   Type: **SSH Host** (automatically filled in)

   Function: **Collect** (clear), **Respond** (select)

   Name: Enter a name for the connector, such as `TCP Reset` for example.

   Tenant Name: Choose a tenant from the drop-down list.

   Run On: Choose a Modular Sensor.

2. Select **Next** and enter the following in the **Configuration** section:

   Hostname: Enter 127.0.0.1

   The connector runs on the selected Modular Sensor, which executes the TCP Reset script locally.

   Port: `22`

   Username: Enter a valid username for the SSH server on the Modular Sensor

   Password: Enter the password associated with the username.

3. Select **Next** and then **Submit** the configuration.

   How It Works

   When a TCP Reset response action is triggered, the Stellar Cyber connector framework initiates a local SSH session to 127.0.0.1 on port 22 and executes the TCP Reset script on the Modular Sensor.

   After the response action initiates, the Modular Sensor sends TCP Reset packets directly onto the network segments it monitors through its reset interface (explained in Step 6 in the following section, “Create an Automated Threat Hunting Playbook”).

   Because the sensor observes both sides of each TCP session, it can accurately identify and terminate malicious or unwanted connections in real time.

   Environment Requirements and Limitations

   To ensure that TCP Reset responses function correctly on your network, verify the following conditions before enabling this feature:

   - Intermediary devices (firewalls, proxies, or similar) might block the TCP RST packets that the Modular Sensor sends.

     The Modular Sensor sends TCP Reset packets to both the victim and attacker. If intermediary devices filter or drop RST packets, the reset action will not terminate the session effectively.

   - There must not be a NAT device that translates the original source IP address.

     Network address translation modifies source IP addresses, which prevents the Modular Sensor from matching packets correctly and identifying the intended connection to reset.

NAT applied to outbound traffic from an internal network to the Internet usually does not interfere, provided that the Modular Sensor observes both directions of the connection before the NAT boundary. However, if the NAT device lies between the sensor and either the victim or attacker, the reset action might not function as intended.

Create an Automated Threat Hunting Playbook

Create an ATH playbook that detects a threat that depends on an active TCP connection and can be mitigated by the termination of the connection by sending TCP resets to the two endpoints. A few examples of this type of threat are brute-force logins, command-and-control callbacks, covert tunneling, data exfiltration, exploit delivery, and large-scale reconnaissance.

1. Navigate to **Automation**, select **Create**, and configure an Automated Threat Hunting (ATH) playbook that detects the type of threats that rely on a TCP connection.

   For information about ATH playbooks, see **Automated Threat Hunting**.

2. Enable the **Calculations** section in your playbook and then select **+ (Add a Calculation)**.

   Alerts that share the same detection engine (`engid`) and the same source host IP address (`srcip`) are likely duplicates. Without deduplication, each duplicate alert can independently trigger a response. This results in unnecessary repeated responses, which you can eliminate by configuring the **Calculations** section.

3. Add the following calculation to deduplicate alerts:

   Name: `find_unique_engid`

   Calculation type: **Top**

   Field to calculate on: **engid**

   Calculate By: **count**

   Size: Enter the size you want to use; for example, `10`.

4. Select **Add Subgroup** and then enter the following:

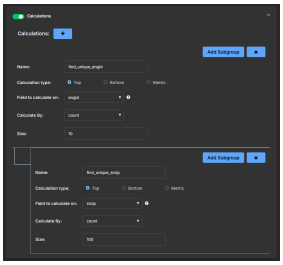
   Name: `find_unique_srcip`

   Calculation type: **Top**

   Field to calculate on: **srcip**

   Calculate By: **count**

   Size: Enter the size you want to use; for example, `100`.



With this calculation, you group alerts by the `engid` and `srcip` fields and return only the most frequent unique values (top values). You first identify the top 10 detection engines that detected the alert (for example, correlation rules, machine learning models, or IDS signatures). Then, within each detection engine, you identify the top 100 alerts that share the same source IP address. This nested grouping ensures that alerts sharing the same detection engine and source IP address are treated as duplicates, so only one response is triggered per unique combination rather than multiple redundant actions.

5. In the **Actions** section, select **Run Script**; and then from the **Target** list, choose the connector you created previously (**TCP Reset**, for example).
6. For **Script to Execute**, choose **custom** and then copy and paste the following script as a single block in the **Script Body** field:

```
{{#ctx.payload.aggregations.find_unique_engid.buckets}}
{{#find_unique_srcip.buckets}}
tcpreset {{engid}} -i eth1 -o eth0 -h {{srcip}} -d 180
{{/find_unique_srcip.buckets}}
{{/ctx.payload.aggregations.find_unique_engid.buckets}}
```



Enter the script exactly as shown. Do not add any blank lines between lines of code.

Lines 1 and 2 and lines 4 and 5 define nested loops that iterate through the results of the **Calculations** step. The outer loop cycles through each detection engine in the `find_unique_engid` group, and the inner loop cycles through each source IP address in the `find_unique_srcip` subgroup within that detection engine. This nested structure ensures that the script processes every unique combination of detection engine and source IP address identified in the **Calculations** step.

Line 3 defines the response by issuing a `tcpreset` command. This command uses the detection engine and source IP address values identified in the **Calculations** step and dynamically substitutes them into the placeholders `{{engid}}` and `{{srcip}}`. The `tcpreset` syntax is as follows:

```
tcpreset {{engid}} -i {sniff intf} -o {reset intf} -h {{srcip}} [-p
{{dstport}}] -d {duration}
```

`-i {sniff intf}` (required): The monitoring interface that listens to traffic. It must be active. You can view available interfaces with the `show interface physical` command. The sensor validates the interface status when receiving the command.

`-o {reset intf}` (required): The interface used to send the TCP reset traffic. It must be active and have a valid IP address. You can also view available interfaces with the `show interface physical` command. The sensor validates the interface status when receiving the command. There is no default interface; you must always specify one. If you want to use the management interface, you must include it explicitly. If you want to use a non-management interface, configure a valid IP address on the interface with the `set interface <name>` command before sending TCP resets through it.

`-d {duration}` (required): The number of seconds that the TCP reset runs. For example, if you set a duration of 180, the sensor continues to monitor the sniffing interface for packets that match the specified criteria (detection engine + source IP

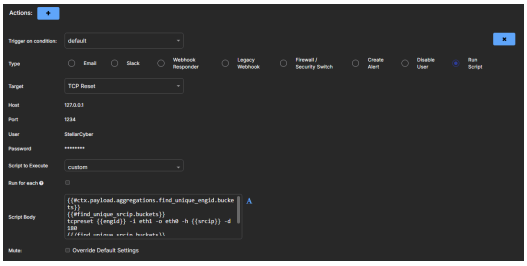
address + optional destination port) for the next 180 seconds. Any further attempt to maintain or establish a TCP session during this time is terminated by immediately sending TCP resets to both ends of the connection.

> Do not change the double curly brace values `{{engid}}` and `{{srcip}}`. The sensor automatically extracts these values from the matching alerts in the playbook.

When this command is executed, the Data Processor sends TCP resets to both sides of the connection through the reset interface on the Modular Sensor. This means that all TCP sessions involving the specified source IP address are terminated, regardless of whether the packets are inbound or outbound. By default, the host is isolated from all TCP connections. To limit the reset to a specific port, add the optional `-p {{dstport}}` parameter:

```
tcpreset {{engid}} -i eth1 -o eth0 -h {{srcip}} -p {{dstport}} -d 180
```

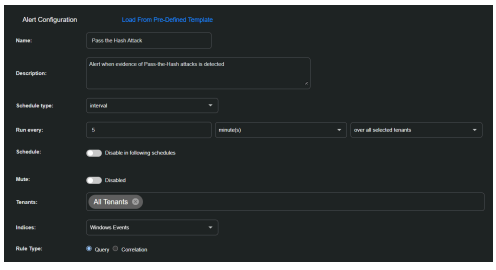
> Do not change the `{{dstport}}` placeholder. With this option, only traffic between the source IP address and the specified destination port is terminated, but resets are still applied to both ends of the connection.



7. To save and activate the ATH playbook, select **Submit**.

Configure the Alert

Configure an alert that runs every five minutes against the Windows Events index to identify suspicious logins that may indicate pass-the-hash activity. Running this check frequently ensures timely detection of credential misuse.

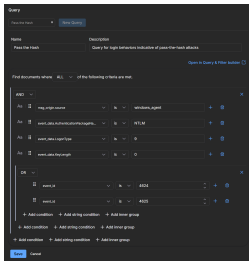


1. Navigate to **Automation | Automation** and click the **Create**button to add a playbook.
2. Enter the following in the **Alert Configuration** section:
    - **Name**: **Pass the Hash Attack**
    - **Description**: **Alert when evidence of Pass-the-Hash attacks is detected**
    - **Schedule type**: **interval**

- **Run every**: **5 minute(s) over all selected tenants**
- **Schedule**: To[illegible] off (default setting)
- **Mute**: Toggle off (default setting)
- **Tenants**: **All Tenants**
- **Indices**: **Windows Events**
- **Rule Type**: **Query**

Build a Query

Build a query that identifies logins using New Technology LAN Manager (NTLM), a legacy Windows authentication protocol. The query focuses on logins that use cached credentials and no session key, which is behavior consistent with common pass-the-hash techniques. The query also includes both successful and failed login attempts to ensure broader visibility into potentially unauthorized access.



1. Select **New Query**.
2. Enter the following:
    - **Query Name**: **Pass the Hash**
    - **Description**: **Query for login behaviors indicative of pass-the-hash attacks**
    - Find documents where **ALL** of the following criteria are met.
    - Select **+ Add condition** and enter **msg_origin.source**, **is**, **windows_agent**.
3. Select **+ Add condition** and enter **event_data.AuthenticationPackageName**, **is**, **NTLM**.
4. Select **+ Add condition** and enter **event_data.LogonType**, **is**, **9**.

    This login type (9) indicates a login using cached credentials.
5. Select **+ Add condition** and enter **event_data.KeyLength**, **is**, **0**.

    This (0) indicates that no session key was requested.
6. Select **+ Add Inner Group**.
7. Change the condition type from **AND** to **OR**.
8. Select **+ Add condition** within the inner group and enter **event_id**, **is**, **4624**.

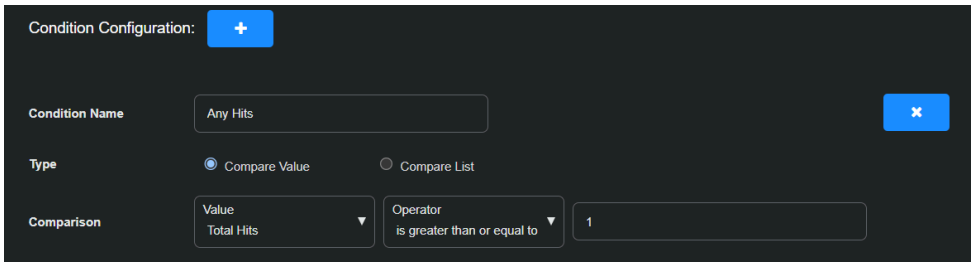
    This indicates a successful login.
9. Still within the inner group, select **+ Add condition** and enter **event_id**, **is**, **4625**.

This indicates a failed login.

10. Select **Save** to save th ery.

Configure a Condition

Define a condition that triggers the playbook actions if at least one matching event is found. This allows the playbook to respond immediately to any signs of pass-the-hash activity.



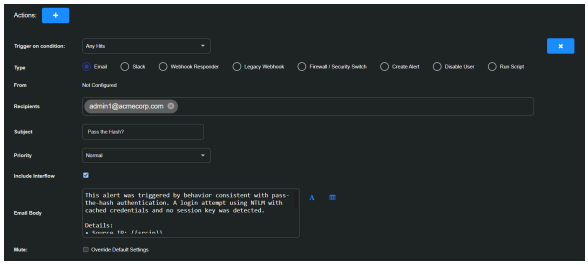
Condition Name: **Any Hits**

Type: **Compare Value**

Comparison: **Total Hits**, **is greater than or equal to**, **1**

Configure an Email Message as the Action

Specify the actions to take when the condition is met. In this example, the playbook sends an email notification that includes relevant details for investigation, including Interflow data to support deeper analysis.



1. Set the **Trigger on condition** to **Any Hits**.
2. Enter the following to set up email messages:
    - **Type**: **Email**
    - **Recipients**: Either type in an email address to use or choose a previously configured recipient from the drop-down list.

      Refer to Adding a Recipient for information on managing recipients globally.
    - **Subject**: **Pass the Hash?**
    - **Priority**: **Normal**
    - **Include Interflow**: (select to attach Interflow records to the email)
    - **Email Body**: Enter the following:

```
This alert was triggered by behavior consistent with pass-the-hash
authentication. A login attempt using NTLM with cached credentials
and no session key was detected.

Details:

    ○ Source IP: {{srcip}}
    ○ Username: {{username}}
    ○ Host: {{hostname}}
    ○ Logon Type: {{logon_type}}
    ○ Event Time: {{timestamp}}

Please review these details in Interflow and investigate whether the
activity is authorized or indicative of credential misuse or lateral
movement.
```

- **Mute**: (disable)

 See Configure Actions for details on configuring each type of action.

Save & Run the Playbook

Select **Submit** to save and immediately run the playbook.

The new playbook appears in the Automated Threat Hunting Playbooks list.

© 2026 Stellar Cyber . All rights reserved.